# Intrusion Detection System Deployment and Packet Analysis Using Suricata

---

## Introduction

With the increasing sophistication of cyber threats, organizations require efficient mechanisms to monitor and detect malicious activities within their networks. Intrusion Detection Systems (IDS) play a critical role in identifying suspicious traffic and generating alerts for further investigation. This project focuses on the deployment of Suricata IDS in a controlled lab environment, simulation of network and application-layer attacks, and analysis of captured traffic using Wireshark.

---

## Aim and Objectives

The aim of this project is to deploy and configure an IDS and analyze malicious network activities.

The objectives include:

- To configure a functional Suricata IDS environment
- To simulate reconnaissance and application-layer attacks
- To detect malicious traffic using IDS rules
- To capture and analyze packets using Wireshark
- To correlate IDS alerts with real network traffic

---

## Lab Setup and Environment

The lab environment consists of:

- Kali Linux (Attacker + IDS + Analysis)
- Metasploitable2 (Vulnerable Target)
- Virtual network configured using NAT

The systems were configured within the same subnet (192.168.100.0/24), enabling communication between attacker and target.

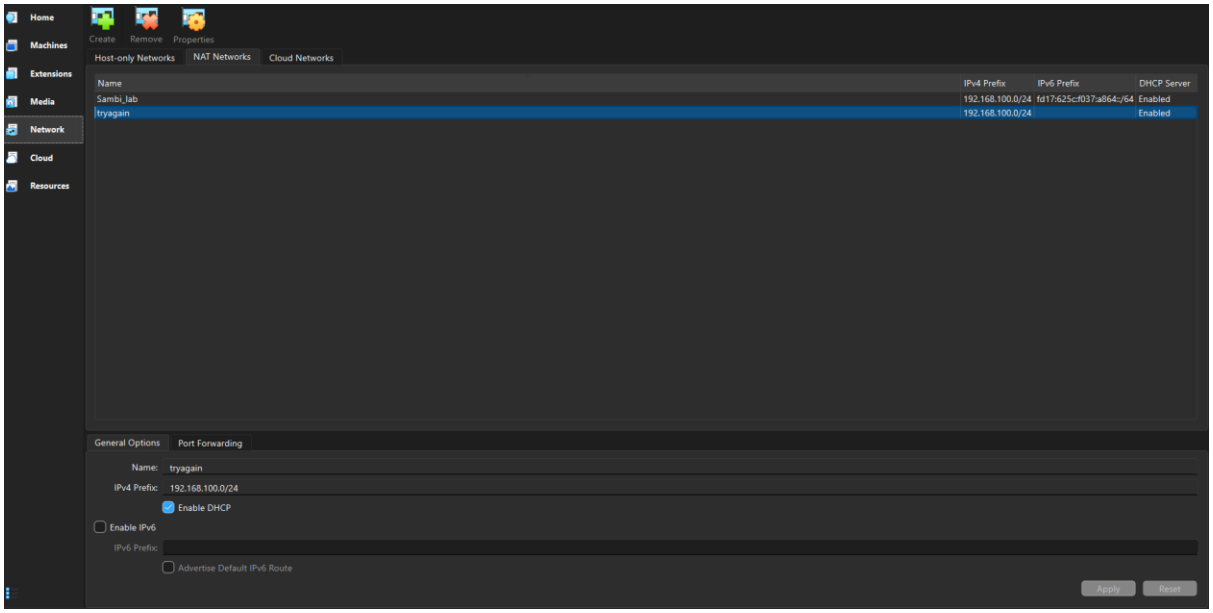

Figure 1: NAT network configuration

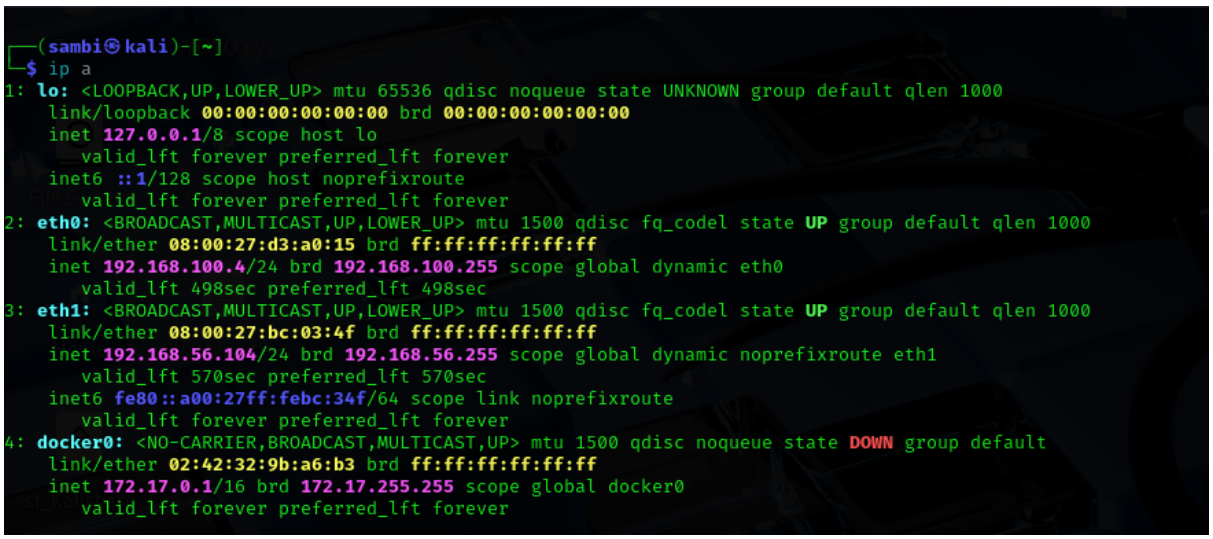

Figure 2: Kali IP configuration

---

## Suricata IDS Deployment

Suricata was installed and configured on Kali Linux. The configuration file (suricata.yaml) was modified to define the local network range:

HOME_NET: "[192.168.100.0/24]"

Suricata was then executed in IDS mode using the command:

sudo suricata -c /etc/suricata/suricata.yaml -i eth0

This enabled real-time monitoring of network traffic on the selected interface.

```
┌──(sambi㉿kali)-[~]
└─$ sudo nano /etc/resolv.conf
[sudo] password for sambi:

┌──(sambi㉿kali)-[~]
└─$ sudo chattr +i /etc/resolv.conf

┌──(sambi㉿kali)-[~]
└─$ ping -c 3 google.com
PING google.com (216.58.205.174) 56(84) bytes of data.
64 bytes from lcmada-ad-in-f14.1e100.net (216.58.205.174): icmp_seq=1 ttl=255 time=132 ms
64 bytes from lcmada-ad-in-f14.1e100.net (216.58.205.174): icmp_seq=2 ttl=255 time=165 ms
64 bytes from lcmada-ad-in-f14.1e100.net (216.58.205.174): icmp_seq=3 ttl=255 time=169 ms

--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2085ms
rtt min/avg/max/mdev = 131.618/155.245/168.755/16.764 ms

┌──(sambi㉿kali)-[~]
└─$

┌──(sambi㉿kali)-[~]
└─$ ping 192.168.100.3
PING 192.168.100.3 (192.168.100.3) 56(84) bytes of data.
64 bytes from 192.168.100.3: icmp_seq=1 ttl=64 time=17.8 ms
64 bytes from 192.168.100.3: icmp_seq=2 ttl=64 time=2.84 ms
64 bytes from 192.168.100.3: icmp_seq=3 ttl=64 time=12.4 ms
64 bytes from 192.168.100.3: icmp_seq=4 ttl=64 time=17.3 ms
64 bytes from 192.168.100.3: icmp_seq=5 ttl=64 time=11.9 ms
^C
--- 192.168.100.3 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4031ms
rtt min/avg/max/mdev = 2.844/12.461/17.806/5.390 ms

┌──(sambi㉿kali)-[~]
└─$ cat /etc/resolv.conf
nameserver 8.8.8.8
nameserver 1.1.1.1
```

Figure 3: DNS configuration

```
┌──(sambi㉿kali)-[~]
└─$ sudo suricata -c /etc/suricata/suricata.yaml -i eth0
i: suricata: This is Suricata version 8.0.2 RELEASE running in SYSTEM mode
i: mpm-hs: Rule group caching - loaded: 114 newly cached: 0 total cacheable: 114
i: threads: Threads created -> W: 2 FM: 1 FR: 1   Engine started.
```

Figure 4: Suricata running on interface eth0

---

## Attack Simulation and Detection

### 5.1 Nmap Scan (Reconnaissance Phase)

Aggressive Full Port Enumeration Scan with Service and OS Detection was performed using:

nmap -A -T4 -p- 192.168.100.3

This scan attempts to identify open ports and services on the target system.

Suricata successfully detected this activity and generated alerts indicating suspicious scanning behavior.

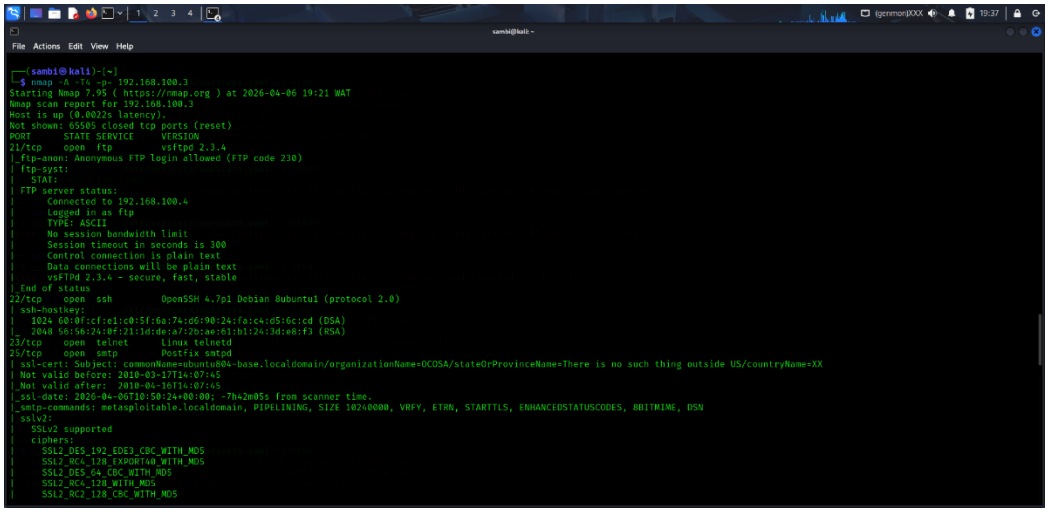


Figure 5: Nmap scan execution

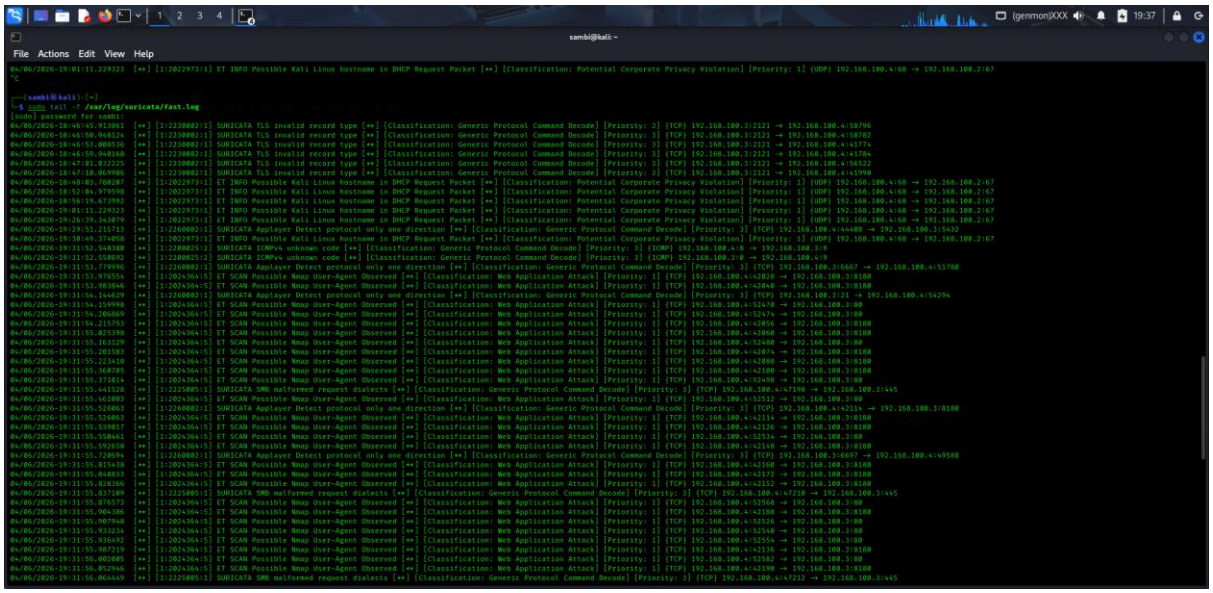


Figure 6: Suricata alert detecting Nmap scan

---

## SQL Injection Simulation

A SQL Injection attack was performed using the DVWA (Damn Vulnerable Web Application) hosted on the target system.

The payload used:

' OR '1'='1' --

This payload manipulates the SQL query to return all records, demonstrating improper input validation.

Although initial attempts produced SQL syntax errors, this confirmed that user input was directly processed by the database, indicating vulnerability.

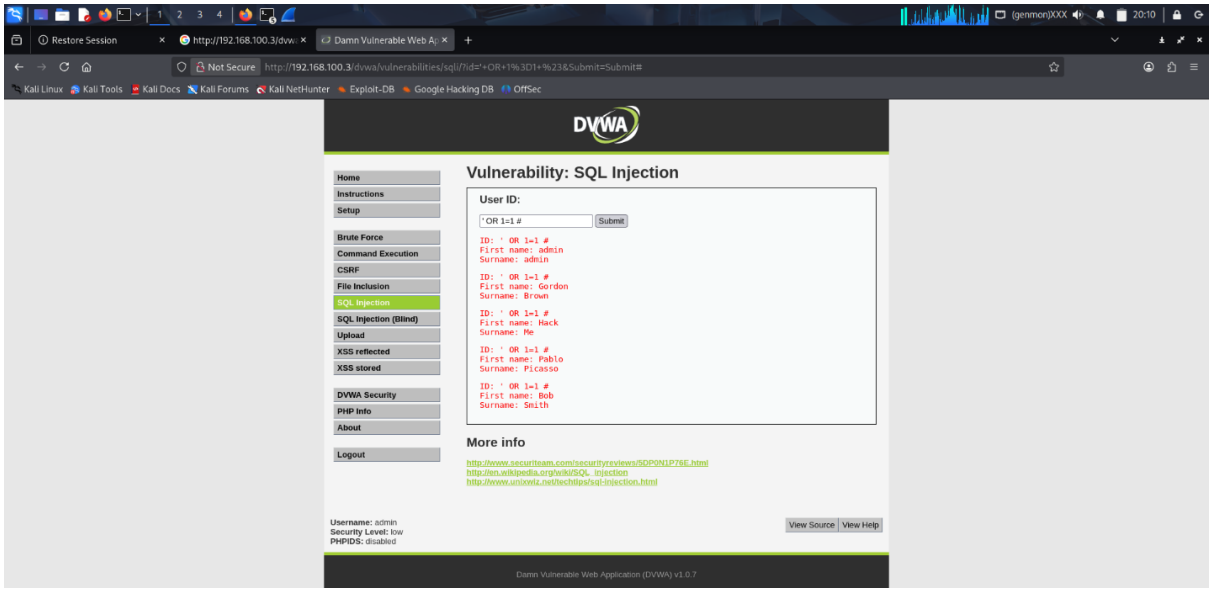


**Figure 7:** SQL Injection Payload Execution – Use of ' OR '1'='1' -- to Manipulate Query Logic

---

## Packet Capture and Analysis (Wireshark)

Wireshark was used to capture and analyze network traffic during the attack simulations.

## Nmap Scan Analysis

Using the filter:

ip.addr == 192.168.100.3

tcp.flags.syn == 1 && tcp.flags.ack == 0

Multiple SYN packets were observed from the attacker to different ports on the target system, confirming a TCP SYN scan.

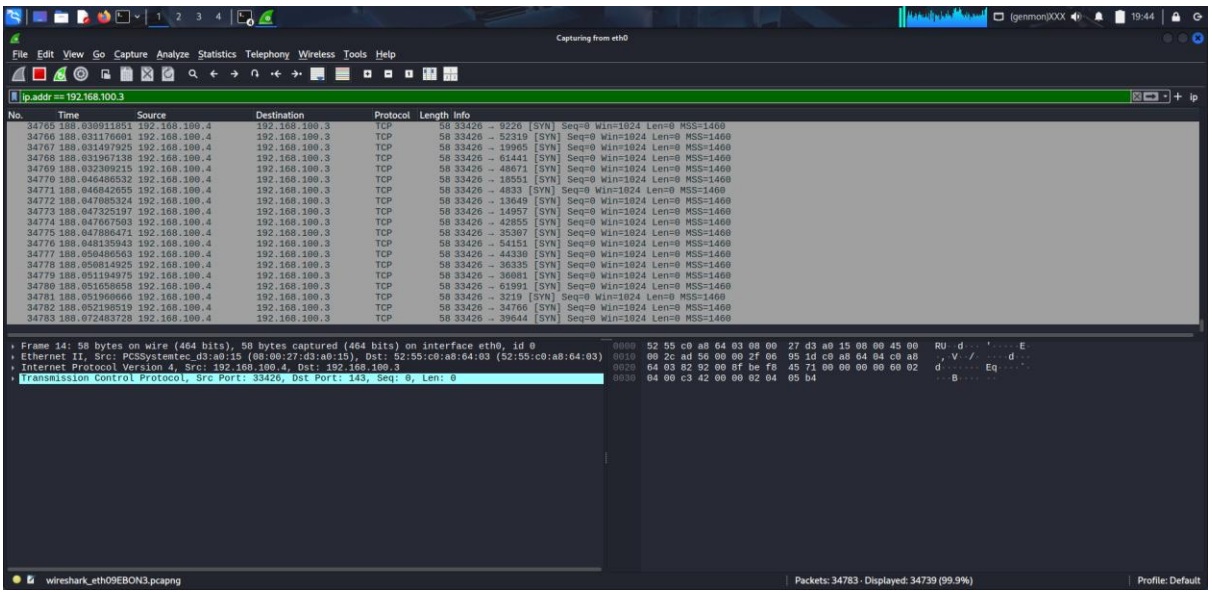


**Figure 8:** Wireshark Packet Capture – TCP SYN Packets Indicating Port Scanning Behavior

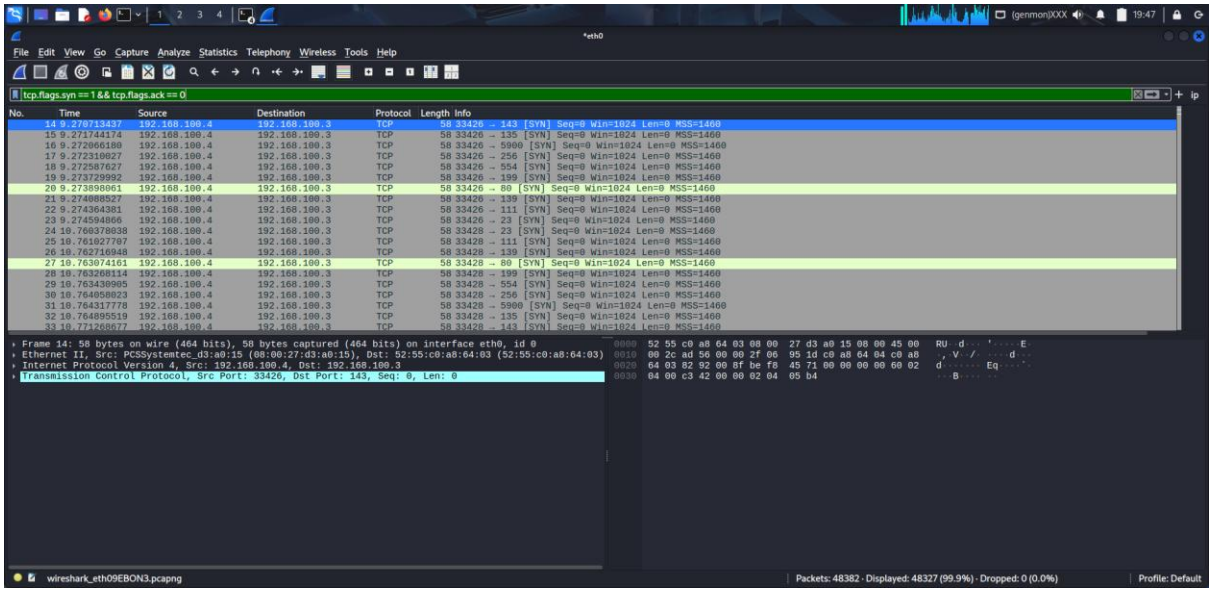


**Figure 9:** Packet Details – TCP Flags Showing SYN Requests Without ACK Responses

## SQL Injection Traffic Analysis

Using the filter:

http

HTTP requests containing the SQL injection payload were identified. The payload was visible within the request, confirming that the malicious input was transmitted over the network.

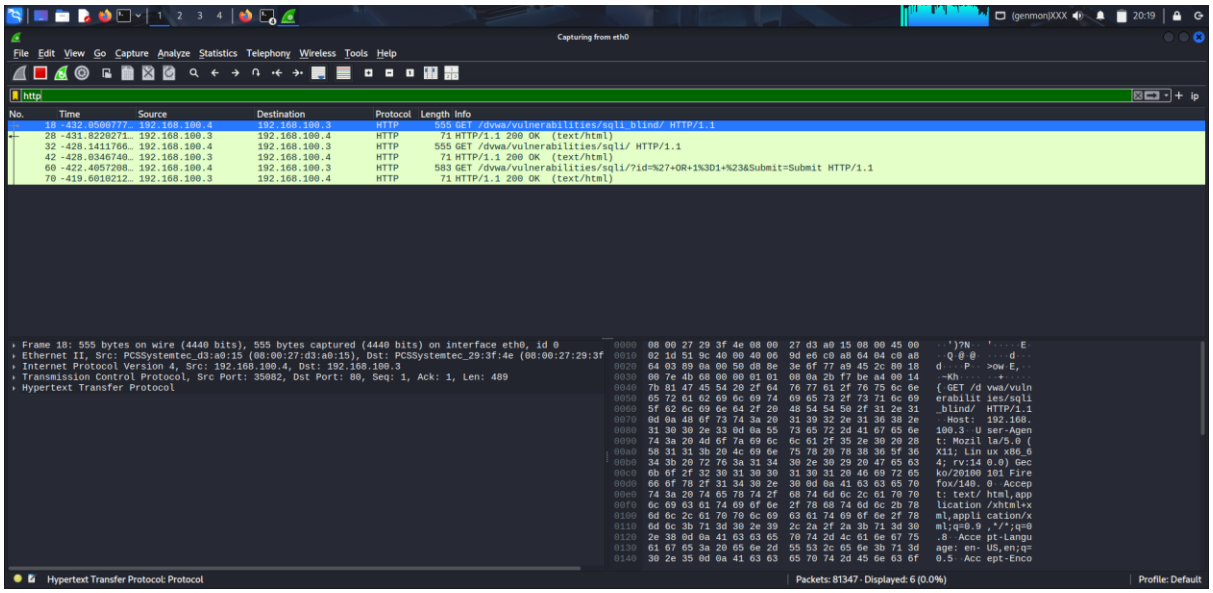


**Figure 10:** Wireshark HTTP Capture – SQL Injection Payload Visible in Request

---

## Correlation of Alerts and Traffic

A key aspect of this project is the correlation between IDS alerts and actual packet data.

- Suricata generated alerts indicating scanning and web-based attacks

- Wireshark confirmed the presence of corresponding network activity
- Source and destination IPs matched across both tools
- Packet payloads validated the nature of the attack

This correlation demonstrates the effectiveness of IDS in detecting and analyzing malicious activities in real time.

---

## Findings

The following observations were made:

- Suricata effectively detected reconnaissance activities such as Nmap scans
- Application-layer attacks such as SQL injection were identifiable through HTTP inspection
- Packet-level analysis provided deeper insight into attack behavior
- Improper input validation in web applications exposes systems to SQL injection attacks

---

## Conclusion

This project successfully demonstrated the deployment and operation of an Intrusion Detection System using Suricata. Through simulated attacks, the IDS was able to detect and generate alerts for malicious activities. Packet analysis using Wireshark further validated these detections by revealing the underlying traffic patterns and payloads.

The integration of IDS alerts with packet-level evidence highlights the importance of combining multiple tools for effective network security monitoring.